

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12401647
Kind Code	B2
Date of Patent	August 26, 2025
Inventor(s)	McCarthy; Michael

Methods and systems for file generation and storage

Abstract

Methods, systems, apparatuses, and non-transitory computer readable media for managing file generation and retention are provided. These systems may be used across a wide range of businesses to significantly automate the generation and storage of files. By utilizing nested file templates as described herein, example systems may increase the ease in which a file template may be generated, stored, retrieved, and retained. Moreover, by using the file retention storage system described below, example systems may simplify the process of ensuring files are retained for an appropriate amount of time and further assuring that files are deleted when they no longer need to be retained.

Inventors:	McCarthy; Michael (Wilmington, DE)
Applicant:	Inkit Worldwide LLC (San Juan, PR)
Family ID:	1000008781668
Assignee:	Inkit Worldwide LLC (San Juan, PR)
Appl. No.:	18/400265
Filed:	December 29, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20250028681 A1	Jan. 23, 2025

Related U.S. Application Data

us-provisional-application US 63515077 20230721

Publication Classification

Int. Cl.: G06F16/00 (20190101); G06F7/14 (20060101); G06F16/13 (20190101); G06F16/16 (20190101); G06F16/17 (20190101); G06F16/955 (20190101); G06F21/62 (20130101); G06F40/186 (20200101); G06F40/40 (20200101); H04L9/40 (20220101); H04L67/06 (20220101)

U.S. Cl.:

CPC H04L63/10 (20130101); G06F7/14 (20130101); G06F16/13 (20190101); G06F16/162 (20190101); G06F16/1734 (20190101); G06F16/9558 (20190101); G06F16/9566 (20190101); G06F21/6209 (20130101); G06F40/186 (20200101); G06F40/40 (20200101); H04L63/08 (20130101); H04L63/0807 (20130101); H04L63/101 (20130101); H04L63/108 (20130101); H04L67/06 (20130101);

Field of Classification Search

CPC: G06F (16/13); G06F (7/14)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
7117436	12/2005	O'Rourke	707/E17.116	G06F 16/958
7627726	12/2008	Chandrasekaran et al.	N/A	N/A
8700576	12/2013	Stakutis et al.	N/A	N/A
9893895	12/2017	Peterson et al.	N/A	N/A
2005/0076066	12/2004	Stakutis et al.	N/A	N/A
2005/0193030	12/2004	Abe	N/A	N/A
2005/0203969	12/2004	Kawabe	N/A	N/A
2009/0248954	12/2008	Hiraiwa et al.	N/A	N/A
2010/0228700	12/2009	Stakutis et al.	N/A	N/A
2012/0179677	12/2011	Roselli	707/736	G06Q 10/10
2014/0129924	12/2013	Le Bescond de Coatpont	715/234	G06F 16/986
2016/0274797	12/2015	Hahn	N/A	G06F 3/0626
2017/0060885	12/2016	Gangadharalah et al.	N/A	N/A
2017/0287090	12/2016	Hunn et al.	N/A	N/A
2018/0005183	12/2017	Hunn	N/A	N/A
2018/0276232	12/2017	Dhanasekaran et al.	N/A	N/A
2020/0327172	12/2019	Coquard	N/A	G06F 16/9035

Primary Examiner: Gebresenbet; Dinku W

Attorney, Agent or Firm: Maynard Nexsen PC

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) The present application claims the benefit and priority of U.S. Provisional Patent Application No. 63/515,077, filed on Jul. 21, 2023,

entitled “Systems and Methods for Document Generation and Automatic Deletion Based on Parameterized Conditions”, the contents of which is hereby incorporated in the entirety.

FIELD

(1) The present disclosure generally relates to data management, and more particularly, methods, systems, apparatuses, and non-transitory computer readable media for file generation, storage, and deletion.

BACKGROUND

(2) For many, if not most, modern-day organizations, an important aspect of their ongoing operations is the creation, communication, and retention of various files to the organization's relevant stakeholders. In addition to communicating information, these files—with regards to both their transmission and their retention—may serve a variety of other purposes, such as formally fulfilling a contractual request or satisfying a regulatory obligation. However, the sheer volume of files, even when restricted to only “formal” files suitable for official communication with external parties, is vast, ranging from contracts and invoices to informational notices and certificates.

Producing such a large volume of files manually would be highly inefficient, consuming valuable human capital on a mundane task while significantly increase the rate of errors. A similar problem arises with regards to ensuring that produced files are retained for an appropriate amount of time and then deleted.

(3) Thus, there is a long sought need to develop systems that are capable of significantly reducing the burden incurred during the process of file generation, storage, and retention—including access control and deletion—are greatly desired.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) The disclosure can be better understood with reference to the following drawings. The elements of the drawings are not necessarily to scale relative to each other, emphasis instead being placed upon clearly illustrating the principles of the disclosure. Furthermore, like reference numerals designate corresponding parts throughout the several views.

(2) FIG. 1 is a block diagram of an exemplary embodiment of a file generation and management system.

(3) FIG. 2 is a block diagram of an exemplary embodiment of a file creation subsystem.

(4) FIG. 3 is a flowchart of an exemplary method of generating files from file templates.

(5) FIG. 4 is a flowchart of an exemplary method of managing the deletion of generated files.

(6) FIG. 5 is a flowchart of a first exemplary method of managing generated files.

(7) FIG. 6 is a flowchart of a second exemplary method of managing generated files.

(8) FIG. 7 is an illustration of the layout of an exemplary file template.

(9) FIG. 8 is an illustration of the data structure of an exemplary file template, such as the exemplary file template of FIG. 7.

(10) FIG. 9 is a flowchart of an exemplary method of generating a file from a file template.

(11) FIG. 10 is a flowchart of an exemplary method of storing a generated file.

(12) FIG. 11 is a flowchart of an exemplary method of controlling access to a stored file.

(13) FIG. 12 is a block diagram of a computing device that may be a client device or server herein.

(14) FIG. 13 is a block diagram of one aspect of a file creation subsystem.

(15) FIG. 14A-B is a flowchart of an exemplary method managing access to generated files.

(16) FIG. 15 is a sequence diagram of one aspect of an orchestrator subsystem.

DETAILED DESCRIPTION

(17) Reference will now be made in detail to exemplary embodiments, examples of which are illustrated in the accompanying drawings. The following description refers to the accompanying

drawings in which the same numbers in different drawings represent the same or similar elements unless otherwise represented. The implementations set forth in the following description of exemplary embodiments do not represent all implementations consistent with the invention. Instead, they are merely examples of apparatuses and methods consistent with aspects related to the invention as recited in the appended claims. Particular aspects of the present disclosure are described in greater detail below. The terms and definitions provided herein control, if in conflict with terms and/or definitions incorporated by reference.

(18) The present disclosure generally pertains to systems and methods for managing file generation and retention. These systems may be used across a wide range of businesses to significantly automate the generation and storage of files. By utilizing nested file templates as described below, systems of the present disclosure may increase the ease in which a file template may be generated. Moreover, by using the file retention and storage system described below, system of the present disclosure may simplify the process of ensuring files are retained for an appropriate amount of time and further assuring that files are deleted when they no longer need to be retained. Thus, the systems and methods here may aid in compliance with various regulations and may aid the protection of confidential information by destroying or safely erasing/removing files after a deletion-triggering event.

(19) For many, if not most, modern-day organizations, an important aspect of their ongoing operations is the creation, communication, and retention of various files to the organization's relevant stakeholders. These files may serve a variety of purposes, such as to communicate relevant information, make some kind of request, or fulfill some kind of legal obligation. For example, retention of compliance files in the financial and health care industry.

(20) The prevalence of these files generates a problem for businesses. Manually entering all relevant customer information is a time-consuming and cost-prohibitive task. Organization's need the ability to automate this process, and to allow for creation of files that will populate fields based on certain conditions. For example, if a file is generated that must also comply with each given State law, then each respective generated file will populate not only the respective fields, but also generate the applicable State portions.

(21) At a high-level, embodiments of the present disclosure may generate a file from a file template in response to receiving a file generation request, among other information possible information, the file generation request may indicate a template to be used and one or more entries of information to be used to populate the file template. As discussed further below, this process may involve generating one or more sub-files—from one or more templates—referred to as sub-templates—via the same process. This process may continue to recursively occur through multiple layers of sub-files (i.e., sub-files of sub-files). Ultimately, the final generated file may be stored in file storage system, where the file is associated with metadata, triggering parameters, and or action items that automatically controls the retention and deletion of the file, as detailed further below.

(22) In a first embodiment a method for generating files with nested templates is disclosed. In one aspect of the first embodiment, a first server may receive a first file template creation request from a client device. In response to receiving the first file template creation request, the first server may generate a first file template with static content data and dynamic content markers. The static content data and the dynamic content data may comprise, amongst other things, positioning information as to position the content at a particular location on a file. For example, the positioning location may provide for an x/y coordinate system to position the content on a text document or a pdf document. In other aspects, it may provide for a pixel location coordinate system. In yet further aspects, metadata from a file may be employed for the files positioning. Even further, Cartesian, cylindrical, and spherical systems may be employed for positioning within a file or document.

(23) Later, in the first embodiment, the first server may receive a first file creation request from the client device. In general, the first file creation request may comprise a set of one or more merge parameter data structures. In response to receiving the first file creation request, the first server a

copy of the first file template with the static content data and the dynamic content markers. In general, the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier. The first server may then insert merge parameter data into the one or more dynamic content markers of the copy of the first file template using a set of one or more merge parameter data structures. In general, the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source and the merge parameter data is obtained from one or more respective merge parameter data sources.

(24) In a second embodiment a computer implemented method for managing generated files is disclosed. In one aspect of the second embodiment, the first server may receive a first file template creation request from a client device. In response to receiving the first file template creation request, the first server may generate a first file template with static content data and dynamic content markers. Later, the first server may receive a first file creation request from the client device. In response to receiving the first file creation request, the first server may generate a first file from the first file template. The first server may then store the first file in non-volatile memory accessible by the first server. Later, the first server may detect a deletion-triggering event. In response to detecting the deletion-triggering event, the first server may delete the first document from the non-volatile memory, or may remove the file from access, or may lock the file and limit it to administrative access only.

(25) In a third embodiment a method for managing access to generated files displayed on a third-party application is disclosed. In one aspect of the third embodiment, a first server may store a first file created from a file template in non-volatile memory accessible by the first server. Later, the first server may receive a request to connect with respect to a first user account of the first server. In general, this request to connect is received from a second server that is associated with a third-party portal. Additionally, the first user account is generally associated with a first user. In response to receiving the request to connect, the first server may authenticate the second server for connecting to the first user account based on the received request to connect with the first user account. In response to authenticating the second server, the first server may generate a first uniform resource identifier (URI) associated with the first file. In general, the first URI links to a first resource comprising the first file.

(26) After generating the first URI, the first server may transmit the first URI and information identifying the first file to the second server. Subsequently, the first server may detect a deletion-triggering event. In response to detecting the deletion-triggering event, the first server may delete the first document from the non-volatile memory.

(27) In a fourth embodiment a method for managing access to generated files is disclosed. In one aspect of the fourth embodiment, a server may store a file in non-volatile memory accessible by the server. Subsequently, the sever may receive from a first client device a request to generate a uniform resource identifier (URI). In general, the first URI is associated with the file, an identity, and a dynamic expire time; In response to receiving the request to generate a URI, the first server generates a uniform resource identifier (URI) associated with the file. In general, the generated URI links to a resource comprising the file and is generated at the server. After generating the URI, the server may, at the request of the first client device, transmit the URI to a second client device.

(28) After transmitting the URI to the second client device, the server may receive from a second client device a request using the URI to request the resource. In response, the server may validate the identity through the server as an authorized identity. To do so, the server may check that the identity is contained in a database of authorized identities in the server. The server may also validate the identity through an identity provider, wherein the identity provider requests the user to login to the identity on an identity hosting platform. The server may then grant or deny access to the resource based on a response from the server and the identity provider.

(29) In a fifth embodiment, a method for generating files with nested templates is disclosed. In one

aspect of the first embodiment, a first file template may be generated. In general, the first file template may comprise static content data and dynamic content markers.

(30) In one aspect of the fifth embodiment, a copy of the first file template may be created. In general, the copy of the first file template may include both the static content data and the dynamic content markers from the first template. The dynamic content markers of the first file template—and thus the copy of these dynamic content markers in the copy of the first file template—may each be associated with a respective merge parameter identifier.

(31) In one aspect of the fifth embodiment, merge parameter data may be inserted at the location of the one or more dynamic content markers of the copy of the first file template. Generally, the merge parameter data is obtained from a set of one or more merge parameter tuples. Each merge parameter tuple may comprise at least a name (merge parameter identifier) and a data type (merge parameter data source), and may comprise formats such as JSON, XML, YAML, to name a few. Each merge parameter tuple may comprise a merge parameter identifier and a merge parameter data source. Merge parameter data may be obtained from a merge parameter data source, and the collective merge parameter data may similarly be obtained from the collective merge parameter data sources of the entire set of one or more merge parameter tuples.

(32) Generally, the dynamic markers are removed from the copy of the first file template, either explicitly or implicitly as part of the insertion process (e.g., the dynamic content markers are replaced).

(33) In an additional aspect of the fifth embodiment, the first file template may be stored in non-volatile memory after it is generated.

(34) In an additional aspect of the fifth embodiment, a file generation request may be received from a client device. Among other potential data, the file generation request may contain a first file template identifier and the set of one or more merge parameter tuples mentioned previously. Generally speaking, the first file template identifier may be associated with the first file template, allowing the request to identify the template the requested file should be based on. Receiving the file generation request may, among other effects, trigger the creation of the copy of the first file template.

(35) In an additional aspect of the fifth embodiment, the copy of the first file request may be transmitted to a first destination. As mentioned above, when the merge parameter data has been inserted into all of the dynamic data markers of the first copy of the first file template, the first file template—which may be thought of as being “filled out”—represents a fully generated file. Because it is usually the fully generated file that is desired, the copy of the first file template is typically only transmitted after the merge parameter data has been inserted into every dynamic content markers of the first file template. The first destination may be associated with a first destination identifier. The first destination identifier may be indicated in the file generation request.

(36) In an additional aspect of the fifth embodiment, the first destination identifier may be associated with native non-volatile memory. Correspondingly, the first copy of the first file template may be stored in the native non-volatile memory after it is generated.

(37) In an additional aspect of the fifth embodiment, the process of inserting the merge parameter data at each one of the one or more dynamic content markers of the copy of the first file template may begin by first identifying a merge parameter tuple from the set of one or more merge parameter tuples. The merge parameter tuple can be identified by comparing each merge parameter tuple's associated merge parameter identifier to the merge parameter identifier associated with a current dynamic content marker. The merge parameter tuple containing a merge parameter identifier matching the merge parameter identifier associated with the current dynamic content marker may then be selected.

(38) Next, merge parameter data may be obtained from the merge parameter data source indicated by the selected merge parameter tuple. Then, the obtained merge parameter data may be inserted at the location of the current dynamic content marker. This process may then be repeated for the next

remaining dynamic content markers that have not yet had merge parameter data inserted.

(39) In an additional aspect of the fifth embodiment, at least of the one or more merge parameter tuples may comprise a merge parameter data source containing a native data payload. Additionally, at least one of the one or more merge parameter tuples may comprise a merge parameter data source containing a remote data pointer.

(40) In an additional aspect of the fifth embodiment, at least a first merge parameter tuple from the identified merge parameter tuples comprises a first merge parameter data source that indicates a second file template. Obtaining the merge parameter data from this first merge parameter data source may be achieved by recursively generating a sub-file. The process of recursively generating the sub-file may start with creating a copy of the second file template. Next, merge parameter data may be inserted at the one or more dynamic content markers of the copy of the second file template using the set of one or more merge parameter tuples. Then, the generated sub-file may be inserted at the associated dynamic content marker of the copy of the first file template.

(41) In an additional aspect of the fifth embodiment, the second file template is associated second file template identifier. The first merge parameter data source may indicate the second file template by identifying the second file template with the second file template identifier.

(42) In a sixth embodiment, a method for managing storage and deletion of generated files is disclosed. In one aspect of the sixth embodiment, a first file may be generated from a first file template. In another aspect of the sixth embodiment, the first file may be stored in non-volatile memory after being generated. In another aspect of the sixth embodiment, the non-volatile memory may be monitored to detect a deletion-triggering event. In another aspect of the sixth embodiment, the first file may be deleted from the non-volatile memory in response to detecting the deletion-triggering event.

(43) In an additional aspect of the sixth embodiment, the deletion-triggering event may comprise a combination of one or more parameters. For example, in one aspect the deletion triggering event may comprise a combination of one or more of (i) time since generating the first file; (ii) user views; (iii) IP address or viewing address; (iv) time spent viewing the first file; or (v) date of expiration; or (vi) response to a biometric scan.

(44) In an additional aspect of the sixth embodiment, the deletion-triggering event may be the passage of a first amount of time since a date associated with the generation of the first file. In an additional aspect of the sixth embodiment, the deletion-triggering event may be an amount of times the first file is accessed exceeds a first access limit threshold. In an additional aspect of the sixth embodiment, the first file template may be generated. For example, in one aspect the first file template may comprise static content data and dynamic content markers. To generate the first file from the first file template, a copy of the first file template may be created. In general, the copy of the first file template may include both the static content data and the dynamic content markers from the first template. The dynamic content markers of the first file template—and thus the copy of these dynamic content markers in the copy of the first file template—may each be associated with a respective merge parameter identifier.

(45) Next, the merge parameter data may be inserted at the location of the one or more dynamic content markers of the copy of the first file template. Generally, the merge parameter data is obtained from a set of one or more merge parameter tuples. Each merge parameter tuple may comprise a merge parameter identifier and a merge parameter data source. Merge parameter data may be obtained from a merge parameter data source, and the collective merge parameter data may similarly be obtained from the collective merge parameter data sources of the entire set of one or more merge parameter tuples.

(46) In a seventh embodiment, a method for controlling access to generated files is disclosed. In one aspect, a first file may be generated from a first nested file template. In one aspect of the seventh embodiment, the first file may be stored in non-volatile memory. In one aspect of the seventh embodiment, a first uniform resource identifier (URI) associated with the first file may be

generated. In general, the first URI may link to a first resource comprising the first file at a data repository or memory location. Continuing, a first URI—along with information identifying the first file—may be provided to an authenticated third-party portal.

(47) In an additional aspect of the seventh embodiment, the URI may be used to access the first file. In particular, a request for the first resource identified by the first URI may be received from the authenticated third-party portal. In general, the request for the first resource may be transmitted in response to input from a first user that is provided to the authenticated third-party portal for accessing the first file. Next, the first resource may be provided to the first user in response to receiving the request for the first resource. In general, providing the first resource to the first user comprises providing the first file to the first user.

(48) In an additional aspect of the seventh embodiment, the third-party portal may be authenticated with respect to a first user account associated with the first user. To start, a request to authenticate the third-party portal with respect to the first user may be received from the third-party portal. In general, the request to authenticate comprises authentication information derived from local login credentials associated with the first user account. Next, the authentication information may be analyzed to verify that the authentication information is derived from the correct login credentials associated with the first user account. Then, in response to verifying the authentication information, a response may be transmitted to the third-party portal indicating that the third-party portal is authenticated with respect to the first user.

(49) In an additional aspect of the seventh embodiment, the response transmitted to the third-party portal may comprise an authentication token for the first user account.

(50) In an additional aspect of the seventh embodiment, the provided information identifying the first file may comprise a title associated with the first file. Additionally, the input from the first user that is provided to the authenticated third-party portal for accessing the first file may comprise input from the first user selecting a displayed title associated with the first user that is hyperlinked to the first URI.

(51) In an additional aspect of the seventh embodiment, the first nested file template may be generated. In an additional aspect of the seventh embodiment, the first nested file template may comprise static content data and dynamic content markers. Additionally, the first file may be generated from the first nested file template in a process starting with creating a copy of the first nested file template with the static content data and the dynamic content markers. In general, the one or more dynamic content markers of the first nested file template may each associated with a respective merge parameter identifier. Next, the merge parameter data may be inserted into the one or more dynamic content markers of the copy of the first nested file template using a set of one or more merge parameter tuples. In general, the set of one or more merge parameter tuples may each comprise a respective merge parameter identifier and a respective merge parameter data source. Additionally, the merge parameter data may be obtained from one or more respective merge parameter data sources.

(52) Referring now to FIG. 1, a block diagram of an exemplary embodiment of a file generation and management system. In the example, a file management system **102** is comprised of a template creation subsystem **106**, a file creation subsystem **107**, a storage subsystem **108**, an orchestrator subsystem **105**, and a networking subsystem **104**. These subsystems may further be configured to third-party systems **109** to allow information or processing to occur on other platforms.

(53) Continuing, the template creation subsystem **106** includes processing routines for preparing and inserting a template within a file. This placement may be through tags or various other content markers delineating the insertion point within a file. To provide a file, the file creation subsystem **107** may be employed, in which it processes and generates a file that is formatted for the template creation system **106**. The storage subsystem **108** is a file repository, typically a cloud repository or other data store that allows access to the created files that may or may not include files with generated templates. The orchestrator system **105** serves as the control subsystem tying the various

processing methods together to for a platform that allows the creation of a file, insertion of a template, and storage and linking to the file. Additionally, a networking subsystem **104** may be employed to connect to various other repositories or link to other platforms that may offload processing or storing.

(54) Referring now to FIG. 2, disclosing a block diagram of a file creation subsystem, in accordance with an exemplary embodiment of the present disclosure. As shown by the figure, a file creation subsystem **107** may comprise a dispatch subsystem **204**, a parameter retrieval subsystem **206**, a security subsystem **209**, a file generation subsystem **212**, a flow director subsystem **215**, and a storage subsystem **108**. Also shown as external elements interacting with the file creation subsystem **107** are third-party system **219**, third-party data source **220**, and third-party data destination **221**.

(55) Broadly speaking, the dispatch subsystem functions to coordinate the operation of the file creation subsystem **107**. The dispatch subsystem **204** may also act as the interface between external third-party systems **219** using the file creation subsystem **107** (e.g., for the creation and storage of various files) and the rest of the file creation subsystem. At a high-level, when the dispatch subsystem **204** receives a file generation request, it may interface with the parameter retrieval subsystem **206**, the security subsystem **209**, and the generation subsystem **212** as part of the process of initiating the handling of the request.

(56) With regards to the parameter retrieval subsystem **206**, a file generation request may contain information—referred to as merge parameters—that may be used to fill in a chosen template. In addition to the data be directly included in the file generation request, the file generation request may also specify a source for the indicated data, such as a universal resource identifier (URI). The parameter retrieval subsystem **206** may work to access the indicated URI at the appropriate third-party data source **220**, retrieve the indicated data, and provide the retrieved data to the generation subsystem **212** as fully resolved merge parameter.

(57) With regards to the generation subsystem **212**, a file generation request may contain an identifier for a file template referred to as the file template ID. This indicated template may be used as the basis for generating the requested file. The generation subsystem **212** may use this information to retrieve (a copy of) the indicated template and may then use the merge parameters provided by the dispatch subsystem **204** and the parameter retrieval subsystem **206** to fill in the dynamic elements of the indicated template.

(58) In particular, as shown by FIG. 2, the file generation subsystem **212** may comprise a render engine **213**, a composition engine **214**, and a template database **218**. Broadly speaking, the render engine **213** may work to first retrieve a copy of the template indicated by the file template ID from the template database **218**. The render engine **213** may then proceed to replace the dynamic elements of the template with the matching merge parameters received from the dispatch subsystem **204** and the parameter retrieval subsystem **206**.

(59) After the file is generated by the generation subsystem **212**, the generated file may be provided to the flow director subsystem **215**. The flow director subsystem **215** may then handle the process of transmitting the generated file to its intended destination, repository, or third-party. One such possible destination is the storage subsystem **108**, wherein the storage subsystem if a file repository for temporary or long term storage of files. Another possible destination is one or more external third-party data destinations **221**, such as a cloud service provider, platform partner, or other repository for performing actions or storing files. For files being stored in the storage subsystem **108**, the flow director subsystem may also provide metadata indicating a retention policy for the file. A retention policy, in one example, sets forth triggering events for deletion, removal, or locking of a particular file upon the occurrence of a triggering event.

(60) In particular, as shown by FIG. 2, the file storage subsystem **108** may comprise a storage engine **216** and a file storage **217**. Broadly speaking, the storage subsystem **108** may work to perform two interrelated functions. For one function, the storage subsystem **108** may receive a

generated file from the flow director subsystem **215** and store the file on the file storage **217** alongside metadata indicating a retention policy for the stored file. For the other function, the storage subsystem **108** may periodically scan through the metadata of generated files stored on the file storage **217** to determine if their associated retention metadata indicates a deletion-triggering event has occurred. For generated files where the storage subsystem **108** determines a deletion-triggering event has occurred, the storage engine **216** may proceed to delete the file from the file storage **217**.

(61) Referring now to FIG. 3, a flowchart illustrating a process of generating files from file templates. To start, as shown by block **302** of FIG. 3, the template creation subsystem **106** generates a first file template. After the template creation subsystem **106** generates the first file, as shown by block **303** of FIG. 3, the file creation subsystem **107**, through generation subsystem **112**, creates a copy of the first file template. After the file creation subsystem **107**, through generation subsystem **112**, inserts merge parameter data into the one or more dynamic content markers of the copy of the first file template.

(62) Referring now to FIG. 4, a flowchart illustrating a process of managing the deletion of generated files. To start, as shown by block **402** of FIG. 4, the file creation subsystem **107** generates a first file from a first file template. After the file creation subsystem **107** generates the first file, as shown by block **403** of FIG. 4, the orchestrator subsystem **105** stores the first file in non-volatile memory of the file storage **217** of the storage subsystem **108**. After the orchestrator subsystem **105** stores the first file in non-volatile memory of the file storage **207**, as shown by block **404** of FIG. 4, the storage subsystem **108**, through storage engine **216**, may monitor for and eventually detect a deletion-triggering event. After the storage engine **216** detects a deletion-triggering event, as shown by block **405** of FIG. 4, the storage subsystem **108**, through storage engine **216**, deletes the first document from the non-volatile memory of the file storage **217** of the storage subsystem **108**.

(63) Referring now to FIG. 5, a flowchart illustrating a first process of managing generated files. To start, as shown by block **502** of FIG. 5, the file generation and storage system **102** generates a first file from a first nested file template. After the file generation and storage system **102** generates the first file, as shown by block **503** of FIG. 5, the file generation and storage system **102** stores the first file in non-volatile memory of the file storage **217** of the storage subsystem **108**. After the file generation and storage system **102** stores the first file in non-volatile memory, as shown by block **504** of FIG. 5, the file generation and storage system **102** generates a uniform resource identifier (URI) linking to the first file. After the file generation and storage system **102** generates the URI linking to the first file, as shown by block **505** of FIG. 5, the file generation and storage system **102** provides information identifying the first file and the first URI to an authenticated third-party portal.

(64) Referring now to FIG. 6, a flowchart illustrating a second process of managing generated files. To start, as shown by block **602** of FIG. 6, the file generation and storage system **102** generates a first file from a first nested file template. After the file generation and storage system **102** generates the first file, as shown by block **603** of FIG. 6, the file generation and storage system **102** stores the first file in non-volatile memory of the file storage **217** of the storage subsystem **108**. After the file generation and storage system **102** stores the first file in non-volatile memory, as shown by block **604** of FIG. 6, the file generation and storage system **102** generates a uniform resource identifier (URI) linking to the first file. After the file generation and storage system **102** generates the URI linking to the first file, as shown by block **605** of FIG. 6, the file generation and storage system **102** transmits the generated URI to a first user. After the file generation and storage system **102** transmits the URI to the first user, as shown by block **606** of FIG. 6, the file generation and storage system **102** may receive a request using the first URI. After the file generation and storage system **102** receives the request using the first URI, as shown by block **607** of FIG. 6, the file generation and storage system **102** may assess one or more parameters to verify the validity of the first request. After the file generation and storage system **102** verifies the validity of the received

request, as shown by block **608** of FIG. **6**, the file generation and storage system **102** may provide the first resource to the first user in response to the first request being verified.

(65) Referring now to FIG. **7**, disclosing a schematic illustrating the layout of an exemplary file template. As shown by the figure, a file template may be comprised of one or more static elements, indicated here by text and dotted lines, and one or more dynamic elements, indicated by text in brackets. In the example template illustrated in FIG. **7**, the dynamic elements comprise a dynamic logo, a dynamic address line 1, a dynamic address line 2, a dynamic address line 3, a dynamic salutation, a dynamic name 1, a dynamic name 2, a dynamic name 3, and a dynamic name 4. As also illustrated by the figure, a template may have one or more nested sub-templates, shown in figure three as nested sub-template 1.

(66) Referring now to FIG. **8**, disclosing a schematic illustrating the data structure of an exemplary file template. A file template may comprise any file, such as a text file, a word file, a pdf file, a markdown file, a Microsoft™ based file, a compressed file, or any other file to a standard operating system that is capable of receiving text input. As shown by the example FIG. **8**, a file generation request made comprise various tuples of information, including a request type (i.e., “Request_Type,”), a file template ID (i.e., “Request_Type,”), a security template ID (i.e., “Security_Template_ID,”), a flow template ID (i.e., “Flow_Template_ID,”), a request type (i.e., “Request_Type,”), merge parameters (i.e., “Merge_Parameters,”), security parameters i.e., (“Security_Parameters,”), and flow parameters. (i.e., “Flow_Parameters”).

(67) Referring now to FIG. **9**, a flowchart illustrating a process of generating a file. To start, as shown by block **902** of FIG. **9**, dispatch subsystem **204** receives a file generation request from third-party system. After the dispatch subsystem **204** receives the file generation request, as shown by block **903** of FIG. **9**, the generation subsystem **212** retrieves a copy of the template identified by the file generation request (or the associated data tuple) from the template database **218**. After the render subsystem **212** retrieves the copy of the identified template, as shown by block **904** of FIG. **9**, the render engine **213** modifies the template copy by replacing the dynamic elements of the template with the data from a matching merge parameter tuple. Specifically, for the render engine **213** replaces the next remaining dynamic element of the retrieved file template using a matching one of the merge parameter tuples included in the received file generation request

(68) If the matching merge parameter tuple has a first template identifier as its data source, as shown by block **905** of FIG. **9**, the generation subsystem **212** returns to block **902** to recursively generate the identified nested sub-template. Otherwise, as shown by block **906** of FIG. **9**, the generation subsystem **212** returns to step **904** if there are remaining unreplaced dynamic elements. Otherwise, the method proceeds to block **907**. After all dynamic elements have been replaced, as shown by block **907** of FIG. **9**, the flow director subsystem **215** transmits modified copy of the identified template and stores it as the requested file in either the storage subsystem **108** or the third-party data destination **221**.

(69) Referring now to FIG. **10**, a flowchart of an exemplary method of storing a generated file. To start, as shown by block **1002** of FIG. **10**, the storage system **108** receives a generated file for storage from the flow director subsystem **215**. The storage system **108** also receives information indicating one or more retention parameters for the received file. After the storage system **108** receives a generated file, as shown by block **1003** of FIG. **10**, the storage engine **216** stores the received generated file on the file storage **217** alongside metadata indicated the associated retention parameters. After the storage engine **216** stores it, the received file on the file storage **217**, as shown by block **1004** of FIG. **10**, the storage engine **216** may periodically scan the retention parameters of its stored generated files, including the file just stored, to determine if their retention parameters indicate that a deletion-triggering event has occurred.

(70) If the retention parameters of a stored file indicate a deletion-triggering event has occurred, as shown by block **1005** of FIG. **10**, the method proceeds to block **1007**. Otherwise, as shown by block **1006** of FIG. **10**, if there are remaining stored files the storage engine **216** has queued for

scanning, the method returns to block **1004** for the next remaining stored file to be scanned. Otherwise, the method ends.

(71) If the retention parameters of a stored file indicate a deletion-triggering event has occurred, as shown by block **907**, the storage engine **216** deletes the stored file from the storage engine **216**. A deletion-triggering event may occur for any number of parameters, and including multiple parameters, such as times viewed, and duration of viewing a file. These may be set as parameters or variables and may form an algorithmic means of deletion and retention, such as an if then, or other rules based deterministic system. In one aspect, a deletion/retention parameter may be linked to a device's biometric engine (Face ID; Fingerprint). In other aspects, a deletion/retention parameter may be tied to an IP address, or accessed location, such an aspect may allow deletion if a sensitive file is set with such a parameter as to not open outside a given territory. Thus, this allows for combatting acts by foreign agents, and even nation states by placing security within file retention.

(72) Referring now to FIG. **11**, a flowchart of an exemplary method of controlling access to a stored file. To start, as shown by block **1102** of FIG. **11**, the orchestrator subsystem **105** receives a request from a third-party portal to authenticate the third-party portal with respect to a first user account of the file management system **102**. After the orchestrator subsystem **105** receives the authentication request, as shown by block **1103** of FIG. **11**, the orchestrator subsystem **105** processes the request and, in response to verifying the provided authenticating information, authenticates the third-party portal with respect to the first user account. After the orchestrator subsystem **105** authenticates the third-party portal, as shown by block **1104** of FIG. **11**, the orchestrator subsystem **105** generates one or more URIs associated with one or more generates files. After the orchestrator subsystem **105** generates the one or more URIs, as shown by block **1105** of FIG. **11**, the orchestrator subsystem **105** transmits the generated URIs, along with information identifying the generated files associated with the URIs, to the authenticated third-party portal. After the orchestrator subsystem **105** transmits the URIs and related identifying information to the third-party portal, as shown by block **1106** of FIG. **11**, the orchestrator subsystem **105** receives a request for a resource identified by one of the URIs. After the orchestrator subsystem **105** receives the request for the resource, as shown by block **1107** of FIG. **11**, the orchestrator subsystem **105** provides the first resource in response to the request.

(73) Referring now to FIG. **12**, a block diagram of a computing device. As shown by the figure, a computing device **1202** may comprise a processing system **1203**, a memory system **1206**, a storage system **1209**, a network adapter **1212**, an input/output (IO) interface **1213**, and a bus interface **1214**. In turn, the processing system **1203** may comprise one or more processing units, shown here as processing units **1204**, and **1205**. Similarly, the memory system **1206** may comprise one or more RAM modules, shown here as RAM modules **1207** and **1208**. Likewise, the storage system **1209** may comprise one or more storage drives, shown here as storage drives **1210** and **1211**.

(74) Generally speaking, the IO Interface **1213** may be connected to various peripheral devices, shown here as a display **1216** and an input device **1217**. Similarly, the network adapter **1212** may connect to one or more external networks, shown here as network **1215**.

(75) Referring now to FIG. **13**, a block diagram of one aspect of a file creation subsystem. As shown by the figure a file creation subsystem may comprise a doc gen queue **1302**, a doc gen worker **1303**, a template database **1304**, a conversion queue **1305**, a file conversion worker **1306**, a renders DB **1307**, a temp file storage **1308**, a file composition queue **1309**, and a file composition worker **1310**.

(76) Referring now to FIG. **14AB**, a flowchart of an exemplary method managing access to generated files. In the example, a check subscription **1402** is provided for access to the system, if granted **1403**, proceed to check whether storage is available on the platform **1405**. If the account has access to the features and permissions are accepted specify files or documents and a type of action **1408**. The system verifies permissions based on parameters **1410**. The system verifies documents exist in repository or database **1411**, if no file located **404** provide alert **1411a**.

Otherwise, proceed to check if user/api-user has access level read to related documents folder **1412**. The system performs access level check based on parameters, including credentials submitted **1413**, and if not available **1414** alert user through system, otherwise the system checks whether files or documents have expired **1415**, if document has expired the system performs deletion or erasure **1416**, and may log such triggering-event that set deletion or erasure. Steps **1418-1422** relate to a check of the system as to whether a file or document exists, whether the user session is active, and if the file does not exist prompts to create a new document **1421**. The system checks to see if the request arrives from another platform via an API **1423**, or if originates within system. The system will then send an email user **1426** with access credentials to the file or document **1427**. The user then may access the system file repository to view the file or document **1428**. The system then prompts the main menu **1429** and file access **200**.

(77) FIG. **15** is a sequence diagram of one aspect of an orchestrator subsystem **1501**. In the example, the orchestrator comprises a routine for verifying access control and deletion, amongst other subsystems, including monitoring for customer link clicks **1503**, which in turn provides documents **1504** upon verification of credentials by accessing **1506** document link components **1502**. The document link components provide credential verification **1505** to access the documents, including enforcing retention policies.

(78) In some embodiments, a non-transitory computer-readable storage medium including instructions is also provided, and the instructions may be executed by a device, for performing the above-described methods. Common forms of non-transitory media include, for example, a floppy disk, a flexible disk, hard disk, solid state drive, magnetic tape, or any other magnetic data storage medium, a CD-ROM, any other optical data storage medium, any physical medium with patterns of holes, a RAM, a PROM, and EPROM, a FLASH-EPROM or any other flash memory, NVRAM, a cache, a register, any other memory chip or cartridge, and networked versions of the same. The device may include one or more processors (CPUs), an input/output interface, a network interface, and/or a memory.

(79) The devices, modules, and other functional units described in this disclosure can be implemented by hardware, or software, or a combination of hardware and software. In some embodiments, functions described as being implemented in hardware may instead be implemented in software or a combination of hardware and software. Likewise, in some embodiments, functions described as being implemented in software may instead be implemented in hardware or a combination of hardware and software. If something is implemented by software, it may be stored in a non-transitory computer-readable media, like the computer-readable media described above. Such software, when executed by a processor, may perform the function of the device, module or other functional unit the software is implementing. The above described devices, modules, and other functions units may also be combined or may be further divided into a plurality of sub-units.

(80) In some places reference is made to standards, including standard methods of performing some task. These standards are revised from time to time, and, unless explicitly stated otherwise, reference to standards in this disclosure refer to the most recent published standard as of the time of filing.

(81) Spatially relative terms, such as “under”, “below”, “lower”, “over”, “upper” and the like, may be used herein for ease of description to describe one element or feature's relationship to another when the apparatus is right side up.

(82) When a feature is referred to as being “on” another feature, the feature may be directly on the other feature with no intervening features present or it may be indirectly on the other feature with intervening features being present. In contrast, when a feature is referred to as being “directly on” another feature, the feature is directly on the other feature with no intervening features present. It will also be understood that, when a feature is referred to as being “connected”, “attached” or “coupled” to another feature, the feature may be directly connected, attached or coupled to the other feature with no intervening features present or it may be indirectly connected, attached or

coupled to the other feature with intervening features being present. In contrast, when a feature is referred to as being “directly connected”, “directly attached” or “directly coupled” to another feature, the feature is directly connected, directly attached, or directly coupled to the other feature with not intervening features present.

(83) The terms “about” and “approximately” shall generally mean an acceptable degree of error or variation for the quantity measured given the nature or precision of the measurements. Typical, exemplary degrees of error or variation are within 20%, preferably within 10%, more preferably within 5%, and still more preferably within 1% of a given value or range of values. Numerical quantities given in this description are approximate unless stated otherwise, meaning that the term “about” or “approximately” can be inferred when not expressly stated.

(84) Ordinal numbers or terms such as “first” and “second” are used only to differentiate an entity or operation from another entity or operation, and do not require or imply any actual relationship or sequence between these entities or operations. Thus, a first feature or element could be termed a second feature or element, and similarly, a second feature or element could be termed a first feature or element without departing from the teachings of the present disclosure. Moreover, the words “comprising,” “having,” “containing,” and “including,” and other similar forms are intended to be equivalent in meaning and be open ended in that an item or items following any one of these words is not meant to be an exhaustive listing of such item or items, or meant to be limited to only the listed item or items.

(85) As used herein, unless specifically stated otherwise, the terms “or” and “at least one of” encompasses all possible combinations, except where infeasible. For example, if it is stated that a component may include “A or B,” then, unless specifically stated otherwise or infeasible, the component may include “A,” “B,” or “A and B.” As a second example, if it is stated that a component includes “at least one of A, B, or C,” then, unless specifically stated otherwise or infeasible, the component may include “A,” “B,” “C,” “A and B,” “A and C,” “B and C,” or “A, B, and C.” This same construction applies to longer lists (e.g., “may include A, B, C, or D”).

(86) As used herein, the singular forms “a”, “an,” and “the” are intended to include the plural forms as well, unless the context clearly indicates otherwise.

(87) Any given element or step of the embodiments disclosed above may be embodied in a single element or step or may be embodied in multiple elements or steps. Moreover, any given element or step of the embodiments disclosed above may be combined and embodied in single element or step or may be combined and embodied in multiple elements or steps.

(88) The sequence of steps shown in the various figures are only for illustrative purposes and do not necessarily indicate that embodiments of the present disclosure are limited to any particular sequence of steps. As such, steps performed by various embodiments of the present disclosure can be performed in a different order while implementing the same method.

(89) In the foregoing specification, embodiments have been described with reference to numerous specific details that can vary from implementation to implementation. Certain adaptations and modifications of the described embodiments can be made. Other embodiments can be apparent to those skilled in the art from consideration of the specification and practice of the invention disclosed herein. It is intended that the specification and examples be considered as exemplary only, with a true scope and spirit of the invention being indicated by the following claims. It is also intended that the sequence of steps shown in figures are only for illustrative purposes and are not intended to be limited to any particular sequence of steps. As such, those skilled in the art can appreciate that these steps can be performed in a different order while implementing the same method.

(90) The disclosure provided herein is further described in the following implementation clauses:

(91) Clause 1. A method for generating files with nested templates, comprising: receiving, at a first server, a first file template creation request from a client device; generating on the first server, in response to receiving the first file template creation request, a first file template with one or more

portions of static content data and one or more dynamic content markers; receiving on the first server, a first file creation request from the client device, wherein the first file creation request comprises a set of one or more merge parameter data structures; creating on the first server, in response to receiving the first file creation request, a copy of the first file template with the one or more portions of static content data and the one or more dynamic content markers, wherein: the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier; and inserting merge parameter data into the one or more dynamic content markers of the copy of the first file template using the set of one or more merge parameter data structures, wherein: the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source; and the merge parameter data is obtained from one or more respective merge parameter data sources.

(92) Clause 2. The method of clause 1, wherein: the first file template creation request comprises a first file template identifier; the one or more portions of the static content data comprises positioning information associated with the one or more portions of the static content data; and the one or more dynamic content markers comprises positioning information associated with the one or more dynamic content markers, and merge parameter identifiers associated with the one or more dynamic content markers.

(93) Clause 3. The method of clause 2, the method further comprising storing by the first server the first file template in non-volatile memory associated with the first server.

(94) Clause 4. The method of clause 1, wherein: the first file template is associated with a first file template identifier; and the first file creation request comprises the first file template identifier.

(95) Clause 5. The method of clause 4, the method further comprising transmitting, after inserting the merge parameter data into the one or more dynamic content markers, the copy of the first file template to a first destination, wherein: the first destination is associated with a first destination identifier; and the first file creation request comprises the first destination identifier.

(96) Clause 6. The method of clause 5, wherein the first destination identifier is associated with native non-volatile memory accessible by the first server, the method further comprising storing by the first server the copy of the first file template in non-volatile memory accessible by the first server.

(97) Clause 7. The method of clause 1, wherein inserting the merge parameter data into the one or more dynamic content markers of the copy of the first file template comprises: identifying a merge parameter data structure from the set of one or more merge parameter data structures comprising a merge parameter identifier matching the merge parameter identifier associated with at least one of the one or more dynamic content marker of the copy of the first file template; obtaining merge parameter data from the merge parameter data source that comprises the identified merge parameter data structure; and inserting the obtained merge parameter data into the dynamic content marker of the copy of the first file template.

(98) Clause 8. The method of clause 7, wherein: at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory associated with the first server as the associated merge parameter data source; and at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory not associated with the first server as the associated merge parameter data source.

(99) Clause 9. The method of clause 7, wherein: at least a first merge parameter data structure from the identified merge parameter data structures comprises a first merge parameter data source that indicates a second file template identifier associated with a second file template; obtaining merge parameter data from the first merge parameter data source comprises generating a sub-file by recursively: creating a copy of the second file template; and inserting the merge parameter data into the one or more dynamic content markers of the copy of the second file template using the set of one or more merge parameter data structures; and replacing a dynamic content marker of the copy of the first file template with merge parameter data obtained from the first merge parameter data

source comprises replacing the dynamic content marker of the copy of the first file template with the generated sub-file.

(100) Clause 10. A system for generating files with nested templates, comprising: a first server, wherein the first server is configured to: receive a first file template creation request from a client device; generate, in response to receiving the first file template creation request, a first file template with static content data and one or more dynamic content markers; receive, a first file creation request from the client device, wherein the first file creation request comprises a set of one or more merge parameter data structures; create, in response to receiving the first file creation request, a copy of the first file template with the static content data and the one or more dynamic content markers, wherein: the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier; and inserting merge parameter data into the one or more dynamic content markers of the copy of the first file template using a set of one or more merge parameter data structures, wherein: the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source; and the merge parameter data is obtained from one or more respective merge parameter data sources.

(101) Clause 11. The system of clause 10, wherein: the template generation request comprises a first file template identifier; the static content data further comprises positioning information; and the one or more dynamic content markers comprises positioning information, and merge parameter identifiers.

(102) Clause 12. The system of clause 11, wherein the first server is further configured to store the first file template in non-volatile memory associated with the first server.

(103) Clause 13. The system of clause 10, wherein: the first file template is associated with a first file template identifier; and the first file creation request comprises the first file template identifier.

(104) Clause 14. The system of clause 13, wherein: the first server is further configured to transmit, after inserting the merge parameter data into the one or more dynamic content markers, the copy of the first file template to a first destination; the first destination is associated with a first destination identifier; and the file generation request comprises the first destination identifier.

(105) Clause 15. The system of clause 14 wherein: the first destination identifier is associated with native non-volatile memory accessible by the first server; and the first server is further configured to store the copy of the first file template in non-volatile memory accessible by the first server.

(106) Clause 16. The system of clause 10, wherein inserting the merge parameter data into the one or more dynamic content markers of the copy of the first file template comprises: identifying a merge parameter data structure from the set of one or more merge parameter data structures comprising a merge parameter identifier matching the merge parameter identifier associated with at least one of the one or more dynamic content marker of the copy of the first file template; obtaining the merge parameter data from the merge parameter data source that comprises the identified merge parameter data structure; and inserting the obtained merge parameter data into the dynamic content marker of the copy of the first file template.

(107) Clause 17. The system of clause 16, wherein: at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory associated with the first server as the associated merge parameter data source; and at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory not associated with the first server as the associated merge parameter data source.

(108) Clause 18. The system of clause 16, wherein: at least a first merge parameter data structure from the identified merge parameter data structures comprises a first merge parameter data source that indicates a second file template identifier associated with a second file template; obtaining merge parameter data from the first merge parameter data source comprises generating a sub-file by recursively: creating a copy of the second file template; and inserting the merge parameter data into the one or more dynamic content markers of the copy of the second file template using the set

of one or more merge parameter data structures; and replacing a dynamic content marker of the copy of the first file template with merge parameter data obtained from the first merge parameter data source comprises replacing the dynamic content marker of the copy of the first file template with the generated sub-file.

(109) Clause 19. A non-transitory computer-readable medium comprising instructions that, when executed by one or more processors, cause the one or more processors to generating files by: receiving, at a first server, a first file template creation request from a client device; generating on the first server, in response to receiving the first file template creation request, a first file template with static content data and dynamic content markers; receiving on the first server, a first file creation request from the client device, wherein the first file creation request comprises a set of one or more merge parameter data structures; creating on the first server, in response to receiving the first file creation request, a copy of the first file template with the static content data and the dynamic content markers, wherein: the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier; and inserting merge parameter data into the one or more dynamic content markers of the copy of the first file template using a set of one or more merge parameter data structures, wherein: the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source; and the merge parameter data is obtained from one or more respective merge parameter data sources.

(110) Clause 20. The non-transitory computer-readable medium of clause 19, wherein: the template creation request comprises a first file template identifier; the static content data comprises positioning information; and the one or more dynamic content markers comprises positioning information and merge parameter identifiers.

Claims

1. A method for generating files with nested templates, comprising: receiving, at a first server, a first file template creation request from a client device; generating on the first server, in response to receiving the first file template creation request, a first file template with one or more portions of static content data and one or more dynamic content markers; receiving on the first server, a first file creation request from the client device, wherein the first file creation request comprises a set of one or more merge parameter data structures; creating on the first server, in response to receiving the first file creation request, a copy of the first file template with the one or more portions of static content data and the one or more dynamic content markers, wherein: the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier; and inserting merge parameter data into the one or more dynamic content markers of the copy of the first file template using the set of one or more merge parameter data structures, wherein: the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source; and the merge parameter data is obtained from one or more respective merge parameter data sources, wherein the merge parameter data obtained from one or more respective merge parameter data sources replaces at least one of the one or more dynamic content markers of the copy of the first file template; determining one or more unreplaced dynamic content markers, wherein the one or more unreplaced dynamic content markers comprise at least one of the one or more dynamic content markers of the copy of the first file template that are not replaced by the merge parameter data; determining additional merge parameter data; and replacing the one or more unreplaced dynamic content markers of the copy of the first file template with the additional merge parameter data.

2. The method of claim 1, wherein: the first file template creation request comprises a first file template identifier; the one or more portions of the static content data comprises positioning information associated with the one or more portions of the static content data; and the one or more

dynamic content markers comprises positioning information associated with the one or more dynamic content markers, and merge parameter identifiers associated with the one or more dynamic content markers.

3. The method of claim 2, the method further comprising storing by the first server the first file template in non-volatile memory associated with the first server.

4. The method of claim 1, wherein: the first file template is associated with a first file template identifier; and the first file creation request comprises the first file template identifier.

5. The method of claim 4, the method further comprising transmitting, after inserting the merge parameter data into the one or more dynamic content markers, the copy of the first file template to a first destination, wherein: the first destination is associated with a first destination identifier; and the first file creation request comprises the first destination identifier.

6. The method of claim 5, wherein the first destination identifier is associated with native non-volatile memory accessible by the first server, the method further comprising storing by the first server the copy of the first file template in non-volatile memory accessible by the first server.

7. The method of claim 1, wherein inserting the merge parameter data into the one or more dynamic content markers of the copy of the first file template comprises: identifying a merge parameter data structure from the set of one or more merge parameter data structures comprising a merge parameter identifier matching the merge parameter identifier associated with at least one of the one or more dynamic content marker of the copy of the first file template; obtaining merge parameter data from the merge parameter data source that comprises the identified merge parameter data structure; and inserting the obtained merge parameter data into the dynamic content marker of the copy of the first file template.

8. The method of claim 7, wherein: at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory associated with the first server as the associated merge parameter data source; and at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory not associated with the first server as the associated merge parameter data source.

9. The method of claim 7, wherein: at least a first merge parameter data structure from the identified merge parameter data structures comprises a first merge parameter data source that indicates a second file template identifier associated with a second file template; obtaining merge parameter data from the first merge parameter data source comprises generating a sub-file by recursively: creating a copy of the second file template; and inserting the merge parameter data into the one or more dynamic content markers of the copy of the second file template using the set of one or more merge parameter data structures; and replacing a dynamic content marker of the copy of the first file template with merge parameter data obtained from the first merge parameter data source comprises replacing the dynamic content marker of the copy of the first file template with the generated sub-file.

10. A system for generating files with nested templates, comprising: a first server, wherein the first server is configured to: receive a first file template creation request from a client device; generate, in response to receiving the first file template creation request, a first file template with static content data and one or more dynamic content markers; receive, a first file creation request from the client device, wherein the first file creation request comprises a set of one or more merge parameter data structures; create, in response to receiving the first file creation request, a copy of the first file template with the static content data and the one or more dynamic content markers, wherein: the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier; and inserting merge parameter data into the one or more dynamic content markers of the copy of the first file template using a set of one or more merge parameter data structures, wherein: the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source; and the merge parameter data is obtained from one or more respective merge parameter

data sources, wherein the merge parameter data obtained from one or more respective merge parameter data sources replaces at least one of the one or more dynamic content markers of the copy of the first file template; determining one or more unreplaced dynamic content markers, wherein the one or more unreplaced dynamic content markers comprise at least one of the one or more dynamic content markers of the copy of the first file template that are not replaced by the merge parameter data; determining additional merge parameter data; and replacing the one or more unreplaced dynamic content markers of the copy of the first file template with the additional merge parameter data.

11. The system of claim 10, wherein: the template generation request comprises a first file template identifier; the static content data further comprises positioning information; and the one or more dynamic content markers comprises positioning information, and merge parameter identifiers.

12. The system of claim 11, wherein the first server is further configured to store the first file template in non-volatile memory associated with the first server.

13. The system of claim 10, wherein: the first file template is associated with a first file template identifier; and the first file creation request comprises the first file template identifier.

14. The system of claim 13, wherein: the first server is further configured to transmit, after inserting the merge parameter data into the one or more dynamic content markers, the copy of the first file template to a first destination; the first destination is associated with a first destination identifier; and the file generation request comprises the first destination identifier.

15. The system of claim 14 wherein: the first destination identifier is associated with native non-volatile memory accessible by the first server; and the first server is further configured to store the copy of the first file template in non-volatile memory accessible by the first server.

16. The system of claim 10, wherein inserting the merge parameter data into the one or more dynamic content markers of the copy of the first file template comprises: identifying a merge parameter data structure from the set of one or more merge parameter data structures comprising a merge parameter identifier matching the merge parameter identifier associated with at least one of the one or more dynamic content marker of the copy of the first file template; obtaining the merge parameter data from the merge parameter data source that comprises the identified merge parameter data structure; and inserting the obtained merge parameter data into the dynamic content marker of the copy of the first file template.

17. The system of claim 16, wherein: at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory associated with the first server as the associated merge parameter data source; and at least of the one or more merge parameter data structures comprises a data payload stored in non-volatile memory not associated with the first server as the associated merge parameter data source.

18. The system of claim 16, wherein: at least a first merge parameter data structure from the identified merge parameter data structures comprises a first merge parameter data source that indicates a second file template identifier associated with a second file template; obtaining merge parameter data from the first merge parameter data source comprises generating a sub-file by recursively: creating a copy of the second file template; and inserting the merge parameter data into the one or more dynamic content markers of the copy of the second file template using the set of one or more merge parameter data structures; and replacing a dynamic content marker of the copy of the first file template with merge parameter data obtained from the first merge parameter data source comprises replacing the dynamic content marker of the copy of the first file template with the generated sub-file.

19. A non-transitory computer-readable medium comprising instructions that, when executed by one or more processors, cause the one or more processors to generating files by: receiving, at a first server, a first file template creation request from a client device; generating on the first server, in response to receiving the first file template creation request, a first file template with static content data and dynamic content markers; receiving on the first server, a first file creation request from the

client device, wherein the first file creation request comprises a set of one or more merge parameter data structures; creating on the first server, in response to receiving the first file creation request, a copy of the first file template with the static content data and the dynamic content markers, wherein: the one or more dynamic content markers of the first file template are each associated with a respective merge parameter identifier; and inserting merge parameter data into the one or more dynamic content markers of the copy of the first file template using a set of one or more merge parameter data structures, wherein: the set of one or more merge parameter data structures each comprise a respective merge parameter identifier and a respective merge parameter data source; and the merge parameter data is obtained from one or more respective merge parameter data sources, wherein the merge parameter data obtained from one or more respective merge parameter data sources replaces at least one of the one or more dynamic content markers of the copy of the first file template; determining one or more unreplaced dynamic content markers, wherein the one or more unreplaced dynamic content markers comprise at least one of the one or more dynamic content markers of the copy of the first file template that are not replaced by the merge parameter data; determining additional merge parameter data; and replacing the one or more unreplaced dynamic content markers of the copy of the first file template with the additional merge parameter data.

20. The non-transitory computer-readable medium of claim 19, wherein: the template creation request comprises a first file template identifier; the static content data comprises positioning information; and the one or more dynamic content markers comprises positioning information and merge parameter identifiers.
